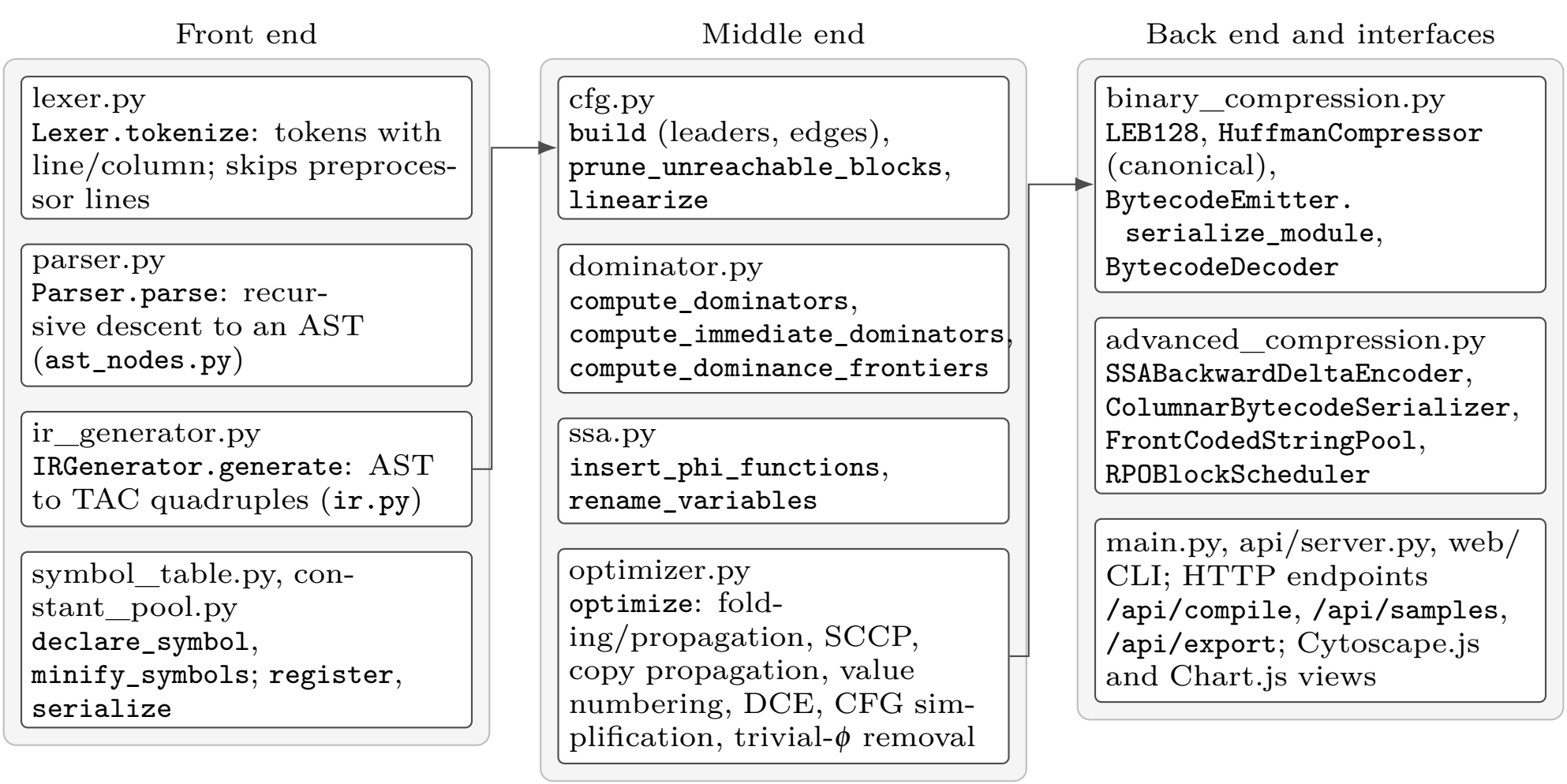
Front end
lexer.py
Lexer.tokenize: tokens with line/column; skips preprocessor lines
parser.py
Parser.parse: recursive descent to an AST (ast_nodes.py)
ir_generator.py
IRGenerator.generate: AST to TAC quadruples (ir.py)
symbol_table.py, constant_pool.py
declare_symbol, minify_symbols; register, serialize
Middle end
cfg.py
build (leaders, edges), prune_unreachable_blocks, linearize
dominator.py
compute_dominators, compute_immediate_dominators, compute_dominance_frontiers
ssa.py
insert_phi_functions, rename_variables
optimizer.py
optimize: folding/propagation, SCCP, copy propagation, value numbering, DCE, CFG simplification, trivial-ϕ removal
Back end and interfaces
binary_compression.py
LEB128, HuffmanCompressor (canonical), BytecodeEmitter. serialize_module, BytecodeDecoder
advanced_compression.py
SSABackwardDeltaEncoder, ColumnarBytecodeSerializer, FrontCodedStringPool, RPOBlockScheduler
main.py, api/server.py, web/ CLI; HTTP endpoints /api/compile, /api/samples, /api/export; Cytoscape.js and Chart.js views